

TraceBridge Medical - Horizon POD System

Document Control

Document Type: Software Architecture Description (SAD)

Document Number: SAD-HZ-002

Revision: C (DRAFT)

Date: August 15, 2026

Standard Alignment: IEC 62304:2015

1. System Overview

The Horizon POD System software is an embedded real-time operating system (RTOS) designed to control a continuous subcutaneous insulin infusion pump. The software is classified as Software Safety Class C, as a failure could result in death or serious injury (e.g., severe hypoglycemia due to insulin overdose).

The architecture is divided into three primary subsystems:

1. **Motor Control Subsystem (MCS)**
2. **Sensor & Alarms Subsystem (SAS)**
3. **Communication & Interface Subsystem (CIS)**

2. Software Subsystems and Units

2.1 Motor Control Subsystem (MCS)

The MCS is responsible for the physical actuation of the shape-memory alloy (SMA) wire that drives the pump mechanism.

- **Unit: DoseCalculator:** Receives target basal rates and bolus requests, converting them into precise micro-step actuation counts.
- **Unit: SMA_Driver:** Generates the specific PWM (Pulse Width Modulation) signals required to heat and contract the SMA wire.
- **Unit: Hardware_Watchdog_Interface:** Periodically pings the independent hardware watchdog to confirm the main thread is not deadlocked.

2.2 Sensor & Alarms Subsystem (SAS)

The SAS monitors the physical state of the POD and alerts the user to adverse conditions.

- **Unit: Occlusion_Monitor:** Reads data from the I2C pressure sensor. Applies a moving average filter to detect sustained pressure spikes indicating a blocked cannula.
- **Unit: Reservoir_Monitor:** Tracks the total insulin delivered vs. the initial 200U fill volume to calculate remaining insulin.
- **Unit: Audio_Transducer:** Generates distinct beep patterns for warnings (e.g., low battery) vs. critical alarms (e.g., occlusion).

2.3 Communication & Interface Subsystem (CIS)

The CIS handles external commands from the patient's smartphone application over Bluetooth Low Energy (BLE).

- **Unit: BLE_Stack:** Implements the standard Bluetooth 5.0 protocol stack.
- **Unit: Command_Parser:** Receives incoming JSON payloads from the mobile app and translates them into internal commands (e.g., START_BOLUS, STOP_BASAL).

3. Cybersecurity and Data Architecture

The device handles Protected Health Information (PHI) and critical life-supporting control commands.

3.1 Authentication

The POD utilizes Out-Of-Band (OOB) Bluetooth pairing. The user must scan a QR code printed on the bottom of the physical POD using the mobile app to establish the initial bond. This prevents unauthorized pairing from nearby devices.

3.2 Data in Transit

The BLE connection relies on standard Bluetooth frequency-hopping spread spectrum (FHSS) to reduce interference.

Design Note: To optimize battery life on the embedded coin cell, we have opted not to implement application-layer AES encryption for the insulin dosage commands or historical logs during transit. We assume that the physical proximity required for BLE (typically <10 meters) provides sufficient mitigation against remote man-in-the-middle attacks.

3.3 Data at Rest

The internal EEPROM stores the last 90 days of insulin delivery history. This data is stored in plain text to allow rapid retrieval by the mobile application during synchronization events.

4. Anomaly Handling and Safe State

If the Command_Parser receives a malformed payload, or if the Hardware_Watchdog_Interface fails to ping within 3 seconds, the software immediately transitions to the **Safe State**.

In the Safe State:

1. All PWM signals to the SMA motor are grounded.
2. A continuous high-pitch alarm is triggered via the Audio_Transducer.
3. The BLE stack broadcasts a CRITICAL_FAULT beacon to the mobile app.

(End of Document)